

Lab05. Provisioning infrastructure.

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning goals

1. Automate creation of VPC.
2. Automate creation of EC2 instance.
3. Automate securing EC2 instance.
4. Provision server.

2 Prerequisites

Remember: all code is prepared for Amazon Linux 2023. If you use Ubuntu in Cloud9 - you have to update commands accordingly.

1. Start your Cloud9 environment.
2. Install Terraform tool: install configuration manager, then add Hashicorp repo, and finally install tool:

```
$ sudo yum install -y yum-utils
$ sudo yum-config-manager --add-repo \
  https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
$ sudo yum -y install terraform
```

3. Check the version of the tool: `terraform version`

3 Exercises

3.1 Initialize Terraform project

In this assignment you will learn how to create an EC2 instance in the default VPC using Terraform.

1. In the `environment/` folder create a subfolder `myproject`. Change directory to `~/environment/myproject/`
2. In terminal, create the new configuration file: `touch main.tf`

3. Paste the following code into `main.tf`. Always replace `<XX>` with your initials (eg. `MyServer<XX>` → `MyServerWT`):

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.1"
    }
  }
  required_version = ">= 1.2.0"
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "app_server" {
  ami          = "ami-071226ecf16aa7d96"
  instance_type = "t2.micro"

  tags = {
    Name     = "MyServer<XX>"
    Terraform = "true"
  }
}
```

Do not forget to save the file.

4. Initialize repository: `terraform init`
 5. Reformat configuration file using: `terraform fmt`
 6. Validate your configuration: `terraform validate`
 7. Build your infrastructure using command: `terraform apply`
 8. Open EC2 console, and verify if your EC2 instance is provisioned, with the assumed name.
 9. Inspect the current state of your project using: `terraform show`
 10. Check the results:
 - is EC2 `MyServerXX` created and running?
- If not, repeat the step.

3.2 Provision VPC

In this step you will provision a new VPC for your own server. Pay attention, what happens with the EC2 instance created in the previous step.

1. Remove block `resource "aws_instance" "app_server" {...}` from `main.tf` file.
2. Paste the following block at the bottom of the file:

```

module "my_vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "my-vpc-<XX>"
  cidr = "10.0.0.0/16"

  azs          = ["us-east-1b"]
  public_subnets = ["10.0.101.0/24"]

  tags = {
    Terraform = "true"
    Environment = "dev"
  }
}

```

When file is saved, reformat its content.

At this step, you use the `module` instead of `service`. Module will create all elements of VPC: VPC, subnet, internet gateway, routing table, routing rules etc.

3. Before you will use module, you have to install it inside your project: `terraform init`
4. Verify you configuration: `terraform validate`
5. Review the plan for changes: `terraform plan` . Study the output. What resources will be removed? What resources will be created?
6. Save the plan to the file: `terraform plan -out add-VPC`
7. Apply the changes to your cloud: `terraform apply add-VPC`
8. Check the results:
 - is EC2 MyServerXX created and running?
 - is VPC and subnet correctly provisioned and configured?

If not, repeat the step.

9. Review AWS VPC service, and verify if your subnet is correctly provisioned. Make sure that EC2 instance was removed.

3.3 Create a security group

In this step you will configure Security Group for your future EC2 instance.

1. At the bottom, paste commands to create a new security group:

```

resource "aws_security_group" "allow_ssh_http" {
  name          = "allow_ssh_http"
  description   = "Allow SSH and HTTP inbound traffic and all outbound traffic"
  vpc_id       = module.my_vpc.vpc_id

  tags = {
    Name = "allow-ssh-http"
  }
}

```

```
}
```

Pay attention, how the Security Group references the VPC created by module.

2. At the bottom of the file, paste commands to add ingress (incoming) and egress (outgoing) rules to your security group:

```
resource "aws_vpc_security_group_egress_rule" "allow_all_traffic_ipv4" {
  security_group_id = aws_security_group.allow_ssh_http.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1" # all ports
}

resource "aws_vpc_security_group_ingress_rule" "allow_http" {
  security_group_id = aws_security_group.allow_ssh_http.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 8080
  to_port           = 8081
}

resource "aws_vpc_security_group_ingress_rule" "allow_ssh" {
  security_group_id = aws_security_group.allow_ssh_http.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 22
  to_port           = 22
}
```

Pay attention, how Rules references the Security Group. Compare these references with the previous step (reference to module).

3. Validate the modified file, save plan, and apply saved plan:

```
$ terraform fmt
$ terraform validate
$ terraform plan -out add-sg
$ terraform apply add-sg
```

4. Open VPC console, and verify if the Security Group exists, and contains required Rules.

3.4 Provision EC2 instance

In this step you will create a new EC2 instance with Ubuntu 24.04 LTS, and start a simple webserver (using busybox tool).

1. At the bottom, paste commands to create EC2 instance:

```
resource "aws_instance" "tf-web-server" {
  ami           = "ami-084568db4383264d4"
  instance_type = "t2.micro"
  key_name      = "vockey"
  subnet_id     = module.my_vpc.public_subnets[0]
```

```

associate_public_ip_address = "true"
vpc_security_group_ids      = [aws_security_group.allow_ssh_http.id]

user_data = <<-EOF
    #!/bin/bash
    echo "<XX> frontend runs on $(date -u)!" > index.html
    nohup busybox httpd -f -p 8080 &
    EOF

user_data_replace_on_change = true

tags = {
    Name = "My-Terraform-Web-Server"
}
}

```

Review how the EC2 instance is associated with: SSH key, subnet, security group and public IP.

2. Validate, plan, and apply the whole configuration.
3. Open EC2 console, and verify if the instance was created. Write down its public IP address.
4. In the browser open address: `http://<PUBLIC-IP-ADDRESS>:8080/`
5. Connect to EC2 instance from your computer using SecureShell. You can use PuTTY or OpenSSH.

Instance in this lab uses Ubuntu 24.04 LTS image, so the username is **ubuntu**.

Demonstrate web page, SSH connection and EC2 instance to your teacher.

1. Create the file `userdata.sh` . Paste the following code inside:

```

#!/bin/bash
echo "<XX> frontend runs FROM FILE on $(date -u)!" > index.html
nohup busybox httpd -f -p 8080 &

```

2. In the file `main.tf` update line `user_data = ...` to:

```

user_data = file("${path.module}/userdata.sh")

```

3. Format, validate, plan the updated configuration. Review: what resources are removed, what are modified.
4. Apply the changes.
5. Verify if the page is updated.

3.5 Remove the provisioned configuration

In this step you will destroy all elements created previously.

1. Run command: `terraform destroy` to remove all provisioned resources.

2. Verify, if EC2 instance, VPC, and security group were removed.

3.6 At home – Provision environment from your own computer

1. Install the Terraform tool on your computer [1].
2. Create the folder `~\.aws`. In Windows, it is the folder: `C:\Users\<YOUR-USERNAME>\.aws\`, in Linux/Mac it is `/home/<YOUR-USERNAME>/.aws/`.
3. Open “Launch AWS Academy Learner Lab” page, click `AWS Details` button. In the “Cloud access” pane on the right, click button AWS CLI: `Show` (see: Fig.1).

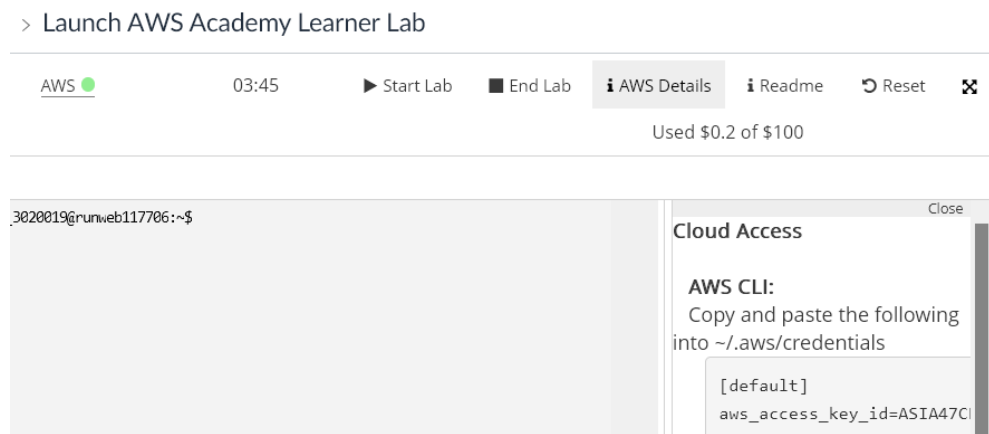


Figure 1: Text field with AWS credentials

4. Copy the content of the gray box, and paste it into `~\.aws\credentials` file. Check twice if the file `credentials` does NOT have extension, eg. `.txt`.
5. Create local Terraform project folder.
6. Copy your `main.tf` file inside.
7. Provision your environment from lab/own computer the same way you did it in the Cloud9.
8. Verify, if your network, security group and EC2 instance are created.
9. Destroy all resources.

Bibliography

- [1] “Install terraform.” [Online]. Available: <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli/>.
- [2] Y. Brikman, *Terraform: Up and running*, 3rd ed. <https://learning.oreilly.com/library/view/terraform-up-and/9781098116736/>; O’Reilly Media, 2022.